



13. 반복자, 함수 객체, 람다와 알고리즘

Heesung Oh

<https://github.com/3dapi>



13. 반복자, 함수 객체, 람다와 알고리즘

- 반복자와 반 개방 구간
- 반복자 범주와 알고리즘 요구 조건
- 반복자 도구와 무효화
- 함수 객체와 호출 가능한 객체
- 람다 표현식과 캡처
- 검색, 판정, 정렬 알고리즘
- 복사, 이동, 변환과 제거 알고리즘
- 수치 알고리즘
- `std::function`과 콜백
- C++20 Ranges
- 알고리즘 선택 기준

컨테이너

↓ 반복자 범위

표준 알고리즘

↓ 호출 가능한 객체

검색·정렬·변환·제거·계산



- 반복자는 범위 안의 원소 위치를 나타내는 객체
- 포인터와 비슷한 연산을 제공
- 반드시 원시 포인터는 아님
- 컨테이너는 저장 구조에 맞는 반복자 타입 제공
- 알고리즘은 반복자가 지원하는 연산만 사용

```
std::vector<int> values{10, 20, 30};  
auto first = values.begin();  
auto last = values.end();  
for (auto itr = first; itr != last; ++itr)  
{  
    std::cout << *itr << ' ';  
}
```

컨테이너 저장 구조



반복자



알고리즘이 사용할 위치 표현



13.1.2 반복 구간과 반 개방 구간

- 표준 알고리즘은 [first, last) 형태 사용
- first가 가리키는 원소는 포함
- last가 가리키는 위치는 포함하지 않음
- end()는 마지막 원소가 아니라 마지막 바로 뒤 위치
- end() 역참조 금지

[10] [20] [30] [끝 위치]
↑ ↑
first last

```
std::vector<int> values{10, 20, 30, 40, 50};
```

```
auto first = values.begin() + 1;  
auto last = values.begin() + 4;
```

```
std::sort(first, last);
```

- 정렬 대상: 인덱스 1, 2, 3
- 빈 범위: first == last



13.1.3 begin, end와 범위

- 수정 가능한 컨테이너의 `begin()`, `end()`는 수정 가능한 반복자 반환
- 상수 컨테이너의 `begin()`, `end()`는 상수 반복자 반환
- `cbegin()`, `cend()`는 항상 상수 반복자 반환

```
std::vector<int> values{10, 20, 30};
```

```
for (auto itr = values.begin(); itr != values.end(); ++itr)
{
    *itr *= 2;
}
```

- 배열과 사용자 정의 범위에는 `std::begin()`, `std::end()` 활용 가능

```
const std::vector<int> fixedValues{10, 20, 30};
auto iterator = fixedValues.begin();
```

```
// *iterator = 100; // 오류
```

```
int values[]{30, 10, 20};
std::sort(std::begin(values), std::end(values));
```



- `const_iterator` 는 원소를 수정하지 못함
- `iterator const` 는 반복자 위치를 바꾸지 못함
- 포인터의 `const int*`, `int* const` 구분과 비슷

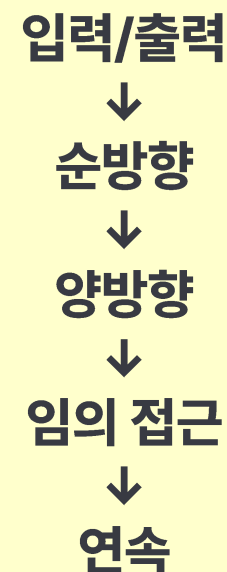
```
std::vector<int> values{10, 20, 30};  
std::vector<int>::const_iterator itr = values.cbegin();  
++itr;  
// *itr = 100; // 오류  
  
std::vector<int>::iterator const fixedltr = values.begin();  
*fixedltr = 100;  
// ++fixedltr; // 오류
```

선언	반복자 이동	원소 수정
<code>iterator</code>	가능	가능
<code>const_iterator</code>	가능	불가
<code>iterator const</code>	불가	가능
<code>const_iterator const</code>	불가	불가



- 반복자는 모두 같은 능력을 제공하지 않음
- 알고리즘은 필요한 이동과 접근 연산을 반복자 범주로 표현
- 요구보다 약한 반복자를 전달하면 알고리즘을 사용할 수 없음

범주	대표 기능	대표 예
입력 반복자	읽기, 단일 순회	<code>istream_iterator</code>
출력 반복자	쓰기, 단일 순회	<code>ostream_iterator</code>
순방향 반복자	앞쪽 이동, 다중 순회	<code>forward_list</code>
양방향 반복자	앞뒤 이동	<code>list</code> , <code>map</code>
임의 접근 반복자	<code>+</code> , <code>-</code> , <code>[]</code>	<code>vector</code> , <code>deque</code>
연속 반복자	연속 메모리 보장	<code>vector</code> , <code>array</code> , <code>string</code>





- 순방향
 - ◆ 앞으로 이동
 - ◆ 같은 범위를 여러 번 순회할 수 있음
 - ◆ 다중 순회 보장
- 양방향
 - ◆ ++, -- 지원
 - ◆ `std::list`, `std::set`, `std::map` 계열

// 순방향 반복자

```
std::forward_list<int> values{10, 20, 30};
```

```
auto first = values.begin();
```

```
auto copy = first;
```

```
++first;
```

```
std::cout << *copy << '\n';
```

// 양방향 반복자

```
std::list<int> values{10, 20, 30};
```

```
auto itr = values.end();
```

```
--itr;
```

```
std::cout << *iterator << '\n';
```




13.2.5 임의 접근 반복자와 연속 반복자

- +, -, +=, -=, [], 비교 연산 지원
- `std::sort()`에 필요

// 임의 접근 반복자

```
std::vector<int> values{10, 20, 30, 40};
```

```
auto iterator = values.begin();
```

```
iterator += 2;
```

```
std::cout << iterator[1] << '\n';
```

// 연속 반복자

```
std::vector<int> values{10, 20, 30};
```

```
auto iterator = values.begin();
```

```
int* pointer = std::to_address(iterator);
```

```
std::cout << pointer[1] << '\n';
```

- 원소가 메모리에 연속 배치
- `std::deque`는 임의 접근 가능하지만 전체 연속 메모리 아님



- `<iterator>` 헤더
- 반복자 이동, 거리 계산, 역방향 순회, 삽입, 스트림 연결 제공
- 반복자 범주에 따라 같은 함수의 비용이 달라짐

도구	역할
<code>std::advance()</code>	반복자를 지정한 거리만큼 이동
<code>std::next()</code>	이동한 새 반복자 반환
<code>std::prev()</code>	뒤로 이동한 새 반복자 반환
<code>std::distance()</code>	두 반복자 사이 거리 계산
<code>rbegin(), rend()</code>	역방향 순회
<code>back_inserter()</code>	<code>push_back()</code> 삽입 반복자
<code>front_inserter()</code>	<code>push_front()</code> 삽입 반복자
<code>inserter()</code>	지정 위치 삽입 반복자
스트림 반복자	입출력을 반복자 범위로 연결





13.3.3 역방향 반복자

- 역방향 반복자에서 ++는 원래 범위의 앞쪽으로 이동
- base()는 대응하는 정방향 반복자 반환
- 역방향 반복자가 가리키는 원소와 base() 위치는 하나 차이

```
std::vector<int> values{10, 20, 30};  
for (auto iterator = values.rbegin();  
     iterator != values.rend();  
     ++iterator)  
{  
    std::cout << *iterator << ' ';  
}
```

// 출력:
// 30 20 10

[10] [20] [30] [끝 위치]
 ↑ ↑
 rbegin rbegin.base()

```
auto reversePosition = std::find(values.rbegin(),  
                                 values.rend(), 20);  
if (reversePosition != values.rend())  
{  
    values.erase(std::prev(reversePosition.base()));  
}
```



- `std::copy()`의 목적지에는 충분한 저장 공간 필요
- 빈 컨테이너에는 삽입 반복자 사용

```
std::vector<int> source{10, 20, 30};  
std::vector<int> destination;  
std::copy(source.begin(), source.end(),  
std::back_inserter(destination));
```

```
std::deque<int> frontValues;  
std::copy(source.begin(), source.end(),  
std::front_inserter(frontValues));
```

```
std::set<int> uniqueValues;  
std::copy(source.begin(), source.end(),  
std::inserter(uniqueValues, uniqueValues.end()));
```

삽입 반복자	사용하는 함수
<code>back_inserter()</code>	<code>push_back()</code>
<code>front_inserter()</code>	<code>push_front()</code>
<code>inserter()</code>	<code>insert()</code>



```
std::istream_iterator<int> first{std::cin};  
std::istream_iterator<int> last;
```

```
std::vector<int> values(first, last);
```

- 기본 생성한 입력 스트림 반복자는 입력 종료 위치
- 입력 실패나 파일 끝에서 끝 반복자와 같아짐

```
std::ostream_iterator<int> output{std::cout,  
", "};  
std::copy(values.begin(), values.end(),  
output);
```

- 두 번째 인수는 각 값 뒤에 출력할 구분 문자열
- 마지막 값 뒤에도 구분 문자열 출력

```
std::transform(  
    std::istream_iterator<int>{std::cin},  
    std::istream_iterator<int>{},  
    std::ostream_iterator<int>{std::cout, " "},  
    [](int value)  
    {  
        return value * value;  
    });
```

- 입력, 변환, 출력을 하나의 범위 처리 흐름으로 연결



13.3.6 반복 중 변경과 반복자 무효화

- 반복자 무효화
→ 반복자가 더 이상 유효한 원소 위치를 나타내지 못하는 상태
- 무효화된 반복자는 비교, 이동, 역참조 모두 위험
- 컨테이너와 연산마다 규칙이 다름

```
std::vector<int> values{1, 2, 3};
```

// 잘못된 코드

```
for (int value : values) {  
    if (value == 2) {  
        values.push_back(20);  
    }  
}
```

- **push_back()** 재할당 시
반복자 전체 무효화 가능

// 순회 중 제거

```
for (auto itr = values.begin(); itr != values.end(); ) {  
    if (*itr % 2 == 0) {  
        itr = values.erase(itr);  
    }  
    else {  
        ++itr;  
    }  
}
```

- **erase()**가 반환한 반복자 사용



- 함수처럼 호출할 수 있는 대상

- ◆ 일반 함수
- ◆ 함수 포인터
- ◆ 함수 객체
- ◆ 람다 표현식
- ◆ 멤버 함수 포인터
- ◆ 멤버 변수 포인터

```
bool IsEven(int value)
{
    return value % 2 == 0;
}
```

```
bool (*predicate)(int) = IsEven;
std::cout << std::boolalpha
    << predicate(10)
    << '\n';
```

```
std::vector<int> values{1, 2, 3, 4, 5};
auto position = std::find_if(values.begin(), values.end(), IsEven);
```

- 표준 알고리즘은 호출 가능한 객체를 비교, 판정, 변환 규칙으로 사용



- `operator()`를 정의하면 객체를 함수처럼 호출 가능
- 함수 객체는 타입과 상태를 가질 수 있음
- 컴파일러가 호출 코드를 인라인화하기 쉬움

```
class IsEven
{
public:
    bool operator()(int value) const
    {
        return value % 2 == 0;
    }
};
```

```
IsEven predicate;
std::cout << std::boolalpha
    << predicate(10)
    << '\n';
```

```
class Printer
{
public:
    void operator()(int value) const
    {
        std::cout << "int: " << value << '\n';
    }

    void operator()(const std::string& value) const
    {
        std::cout << "string: " << value << '\n';
    }
};
```




13.4.3 상태를 가진 함수 객체

```
class IsGreaterThan
{
public:
    explicit IsGreaterThan(int limit) : limit(limit)
    {
    }
    bool operator()(int value) const
    {
        return value > limit;
    }
private:
    int limit;
};
```

- 제한값 20을 객체 내부에 저장
- 전역 변수 없이 조건을 알고리즘에 전달

```
std::vector<int> values{10, 25, 40, 5};
int count = std::count_if( values.begin(),
    values.end(),
    IsGreaterThan{20});
```

```
RunningTotal result = std::for_each(
    values.begin(),
    values.end(),
    RunningTotal{});
```

- for_each()는 내부에서 사용한 함수 객체 반환



- 여러 호출 대상을 하나의 형식으로 호출

```
int Add(int left, int right)
{
    return left + right;
}
int result = std::invoke(Add, 10, 20);
```

```
std::greater<int> greater;
bool first = std::invoke(greater, 20, 10);
```

```
// 멤버 함수 포인터
Player player{100};
auto takeDamage = &Player::TakeDamage;
auto getHp = &Player::GetHp;

std::invoke(takeDamage, player, 30);
std::cout << std::invoke(getHp, player) << '\n';
```

```
// 멤버 변수 포인터
struct Item
{
    int id;
    int price;
};
Item item{1001, 500};

auto price = &Item::price;
std::invoke(price, item) = 700;
```





13.5.2 값 캡처와 참조 캡처

// 값 캡처

```
int limit = 20;  
auto predicate = [limit](int value)  
{  
    return value > limit;  
};  
limit = 100;  
std::cout << std::boolalpha << predicate(50) << '\n';
```

- 람다 객체를 만들 때 값 복사
- 출력 결과는 true

// 참조 캡처

```
int total = 0;  
auto add = [&total](int value)  
{  
    total += value;  
};  
add(10);  
add(20);
```

- 바깥 변수의 실제 값 변경
- 람다가 참조 대상보다 오래 살아남으면 댕글링 참조

```
[=, &total](int value)  
{  
    total += value * multiplier;  
};
```

- 필요한 변수만 명시하는 방식이 안전



13.5.3 초기화 캡처와 mutable

// 초기화 캡처

```
int base = 10;
auto add = [offset = base * 2](int value)
{
    return value + offset;
};
auto pointer = std::make_unique<int>(100);
auto print = [stored = std::move(pointer)]
{
    std::cout << *stored << '\n';
};
```

- 새 멤버를 람다 객체 안에 초기화
- 이동 전용 객체를 람다로 옮길 수 있음

// mutable

```
int count = 0;
auto increment = [count]() mutable
{
    return ++count;
};
std::cout << increment() << '\n';
std::cout << increment() << '\n';
std::cout << count << '\n';
```

- 람다 내부 복사본만 변경
- 바깥 count는 변경되지 않음



13.5.5 반환 형식과 제네릭 람다

// 반환 형식

```
auto absolute = [](int value) {  
    if (value < 0) {  
        return -value;  
    }  
    return value;  
};
```

```
auto convert = [](bool useInteger) -> double {  
    if (useInteger) {  
        return 10;  
    }  
    return 3.5;  
};
```

// 제네릭 람다

```
auto add = [](auto left, auto right) {  
    return left + right;  
};
```

```
std::cout << add(10, 20) << '\n';  
std::cout << add(1.5, 2.5) << '\n';
```

```
auto maximum = [<typename T>(T left, T right) {  
    return left < right ? right : left;  
};
```



13.5.7 클로저 타입과 클로저 객체

```
int offset = 10;  
auto add = [offset](int value)  
{  
    return value + offset;  
};
```

- add는 함수가 아니라 클로저 객체
- 람다 표현식마다 고유한 이름 없는 클래스 타입 생성
- 캡처 값은 클로저 객체의 멤버처럼 저장

```
class AddOffset  
{  
public:  
    explicit AddOffset(int offset)  
        : offset(offset) { }  
    int operator()(int value) const  
    {  
        return value + offset;  
    }  
private:  
    int offset;  
};
```

- 설명을 위한 대응 구조
- 실제 컴파일러 생성 타입과 동일하다는 뜻은 아님

```
auto first = [](int value) { return value * 2; };  
auto second = [](int value) { return value * 2; };
```

- 본문이 같아도 서로 다른
람다 표현식은 다른 타입



13.5.8 람다 캡처와 객체 수명

```
// 안전한 값 캡처
auto MakeAdder(int offset)
{
    return [offset](int value)
    {
        return value + offset;
    };
}
auto addTen = MakeAdder(10);
```

```
// 위험한 참조 캡처 반환
auto MakeInvalidAdder()
{
    int offset = 10;
    return [&offset](int value)
    {
        return value + offset;
    };
}
```

- 함수 종료 후 offset 소멸
- 반환된 람다는 댕글링 참조 보유

```
// this 캡처
auto MakeReader()
{
    return [this]
    {
        return value;
    };
}
```

- 현재 객체 포인터 저장
- 람다보다 객체가 먼저 소멸하면 위험
- C++17 [*this]는 현재 객체를 복사해 캡처



- 범위에서 값을 찾을
- 조건을 만족하는 원소를 찾을
- 조건을 만족하는 원소 수를 셈
- 범위 전체의 논리 상태를 확인
- 선형 검색 알고리즘은 보통 입력 반복자만으로 사용 가능

알고리즘	역할
<code>find()</code>	값과 같은 첫 원소 검색
<code>find_if()</code>	조건을 만족하는 첫 원소 검색
<code>find_if_not()</code>	조건을 만족하지 않는 첫 원소 검색
<code>count()</code>	값과 같은 원소 수
<code>count_if()</code>	조건을 만족하는 원소 수
<code>all_of()</code>	모든 원소가 조건 만족
<code>any_of()</code>	하나 이상 조건 만족
<code>none_of()</code>	조건 만족 원소 없음





```
std::vector<int> values{10, 20, 30, 20};
auto position = std::find(values.begin(), values.end(), 20);
if (position != values.end())
{
    std::cout << *position << '\n';
}

struct Player {
    std::string name;
    int hp;
};

std::vector<Player> players{ {"Knight", 100}, {"Mage", 0}, {"Archer", 70} };
auto position = std::find_if(players.begin(), players.end(),
    [](const Player& player)
    {
        return player.hp == 0;
    });
```

- 찾지 못하면 전달한 끝 반복자 반환
- 부분 범위를 검색했다면 해당 범위의 last와 비교





```
std::vector<int> values{10, 20, 10, 30, 10};  
auto count = std::count(values.begin(), values.end(), 10);  
  
auto aliveCount = std::count_if(players.begin(), players.end(),  
    [](const Player& player)  
    {  
        return player.hp > 0;  
    });  
  
bool allAlive = std::all_of(players.begin(), players.end(),  
    [](const Player& player)  
    {  
        return player.hp > 0;  
    });
```

알고리즘	빈 범위 결과
all_of()	true
any_of()	false
none_of()	true

- 결과가 확정되면 남은 원소를 검사하지 않을 수 있음



- 정렬 알고리즘은 원소 순서를 비교 규칙에 맞게 재배치
- 이진 탐색 알고리즘은 정렬된 범위에서 검색 위치를 빠르게 좁힘
- 정렬에 사용한 비교 규칙과 탐색에 사용한 비교 규칙은 일관되어야 함

알고리즘	역할
<code>sort()</code>	전체 정렬
<code>stable_sort()</code>	안정 정렬
<code>partial_sort()</code>	앞부분만 정렬
<code>nth_element()</code>	n번째 원소 기준 분할
<code>lower_bound()</code>	작지 않은 첫 위치
<code>upper_bound()</code>	큰 첫 위치
<code>equal_range()</code>	동등 범위
<code>binary_search()</code>	존재 여부 반환

- `std::sort()`는 임의 접근 반복자 필요



13.7.1 sort와 stable_sort

```
std::vector<int> values{40, 10, 30, 20};  
std::sort(values.begin(), values.end());
```

```
std::sort(players.begin(), players.end()  
    , [](const Player& left, const Player& right) {  
        return left.score > right.score;  
    });
```

- 비교 함수는 왼쪽 원소가 앞에 와야 하면 true
- 같은 객체끼리 비교했을 때 반드시 false
- 엄격 약순서 만족 필요

```
// 잘못된 비교 함수  
// return left.score >= right.score;
```

```
std::stable_sort(players.begin(), players.end(),  
    [](const Player& left, const Player& right)  
    {  
        return left.score > right.score;  
    });
```

- **stable_sort()**는 동등한 원소의 기존 상대 순서 유지



13.7.3 lower_bound, upper_bound, equal_range

```
std::vector<int> values{10, 20, 20, 20, 30, 40};
```

```
auto lower = std::lower_bound(values.begin(), values.end(), 20);
```

```
auto upper = std::upper_bound(values.begin(), values.end(), 20);
```

```
auto count = std::distance(lower, upper);
```

- [lower, upper)는 값 20과 동등한 원소 범위

```
auto [first, last] = std::equal_range(values.begin(), values.end(), 20);
```

// 정렬된 벡터에 삽입 위치 찾기

```
int value = 25;
```

```
auto position = std::lower_bound(values.begin(), values.end(), value);
```

```
values.insert(position, value);
```

- 위치 탐색은 로그 시간
- 벡터 중간 삽입은 원소 이동 때문에 선형 시간



- 존재 여부만 bool로 반환
- 위치가 필요하면 lower_bound() 사용

```
std::vector<int> values{10, 20, 30, 40};  
bool found = std::binary_search(values.begin(), values.end(), 30);  
  
auto position = std::lower_bound(values.begin(), values.end(), 30);  
if (position != values.end() && *position == 30)  
{  
    std::cout << "found\n";  
}
```

사전 조건

- 범위는 같은 비교 규칙으로 정렬되어 있어야 함
- 내림차순이면 탐색에도 std::greater<> 전달

```
std::sort(values.begin(), values.end(), std::greater<>{});  
bool found = std::binary_search(  
    values.begin(),  
    values.end(),  
    20,  
    std::greater<>{});
```



13.8 복사, 이동, 변환과 제거

알고리즘	역할
<code>copy()</code>	원소 복사
<code>copy_if()</code>	조건 만족 원소 복사
<code>move()</code>	원소 이동
<code>copy_backward()</code>	뒤쪽부터 복사
<code>move_backward()</code>	뒤쪽부터 이동
<code>transform()</code>	원소 변환
<code>remove()</code>	특정 값 논리 제거
<code>remove_if()</code>	조건 만족 원소 논리 제거
<code>unique()</code>	인접 중복 논리 제거
<code>erase_if()</code>	컨테이너에 맞는 조건 제거

- 원소를 다른 범위로 복사
- 원소를 이동
- 값을 변환
- 조건에 맞지 않는 원소를 논리적으로 제거
- 목적지 크기와 범위 중첩 확인 필요



- **std::copy()**

```
std::vector<int> source{10, 20, 30};  
std::vector<int> destination(source.size());  
std::copy(source.begin(), source.end(), destination.begin());  
  
std::vector<int> destination;  
destination.reserve(source.size());
```

- **std::move()**의 두가지 형태

형태	역할
std::move(object)	객체를 우측값으로 변환
std::move(first, last, destination)	범위의 원소들을 이동

```
std::vector<std::string> source{"Sword", "Shield", "Potion"};  
std::vector<std::string> destination(source.size());  
  
std::move(source.begin(), source.end(), destination.begin());
```





13.8.2 transform

- 원소에 변환 함수를 적용, 목적지에 저장
- 둘째 입력 범위가 첫 범위보다 짧으면 범위를 벗어남
- 호출 전에 크기와 목적지 공간 확인

```
std::vector<int> values{1, 2, 3, 4};  
std::vector<int> squares(values.size());
```

```
std::transform(  
    values.begin(),  
    values.end(),  
    squares.begin(),  
    [](int value)  
    {  
        return value * value;  
    });
```

```
// 제자리 변환  
std::transform(  
    values.begin(),  
    values.end(),  
    values.begin(),  
    [](int value)  
    {  
        return value * 2;  
    });
```

```
// 두 입력 범위 결합  
std::transform(  
    left.begin(),  
    left.end(),  
    right.begin(),  
    sums.begin(),  
    std::plus<>{});
```



13.8.3 remove와 remove_if

- 원소 제거
- `std::remove()`는 컨테이너 크기를 줄이지 않음
- 유지할 원소를 앞쪽으로 이동
- 새 논리적 끝 반복자 반환

```
std::vector<int> values{10, 20, 30, 20, 40};  
auto newEnd = std::remove(values.begin(), values.end(), 20);
```

```
// 호출 전  
// 10 20 30 20 40
```

논리 범위
10 30 40 | 미지정된 나머지 영역
↑
newEnd

```
auto newEnd = std::remove_if(  
    values.begin(),  
    values.end(),  
    [](int value)  
    {  
        return value % 2 == 0;  
    });
```

- 알고리즘은 컨테이너 타입을 모름
- 실제 제거는 컨테이너의 `erase()` 필요



- 원소 제거
- 유지할 원소를 앞으로 이동
- 표현이 짧고 컨테이너에 맞는 제거 방법 선택
- 순회 중 직접 제거할 이유가 없다면 std::erase_if()가 명확

// erase-remove 관용구

```
std::vector<int> values{10, 20, 30, 20, 40};  
values.erase(std::remove(values.begin(),  
values.end(), 20), values.end());
```

// 조건 제거

```
values.erase(std::remove_if(  
    values.begin(),  
    values.end(),  
    [](int value)  
    {  
        return value < 0;  
    }  
    ),  
values.end());
```

// C++20

```
std::erase(values, 20);
```

```
std::erase_if(values, [](int value) {  
    return value < 0;  
});
```



13.8.5 unique와 연속 중복 제거

- `std::unique()`는 인접한 동등 원소를 하나로 모음
- 전체 범위에서 같은 값을 모두 찾는 알고리즘이 아님
- 컨테이너 크기를 직접 줄이지 않음

```
std::vector<int> values{10, 10, 20, 20, 10};  
auto newEnd = std::unique(values.begin(), values.end());  
values.erase(newEnd, values.end());
```

```
// 결과  
// 10 20 10
```

- 마지막 10은 앞의 10과 인접하지 않았으므로 남음

```
// 전체 중복 제거  
std::sort(values.begin(), values.end());  
values.erase(std::unique(values.begin(), values.end()), values.end());
```

- 정렬로 같은 값이 인접하게 모임
- 원래 순서를 유지해야 하면 별도 전략 필요



- <numeric> 헤더
- 합계, 내적, 부분 합, 차분, 축약 제공
- 초기값 타입과 연산 순서가 결과 타입과 정밀도에 영향
- accumulate()는 순서 의존 연산에 적합
- reduce()는 연산 순서를 바꿀 수 있음

알고리즘	역할
accumulate()	왼쪽부터 순서대로 누적
inner_product()	두 범위의 대응 원소 결합 후 누적
partial_sum()	앞에서부터 누적한 결과 저장
adjacent_difference()	인접 원소 차이 저장
reduce()	원소 축약
transform_reduce()	변환 후 축약





- 초기값 타입이 누적 결과 타입 결정

```
std::vector<int> values{10, 20, 30};  
int total = std::accumulate(values.begin(), values.end(), 0);
```

- 문자열 연결이나 뺄셈처럼 순서에 민감한 연산에도 사용 가능
- 왼쪽부터 순서대로 누적

```
std::vector<double> values{1.5, 2.5, 3.5};  
double correct = std::accumulate(values.begin(), values.end(), 0.0);  
int truncated = std::accumulate(values.begin(), values.end(), 0);  
int totalPrice = std::accumulate(  
    items.begin(),  
    items.end(),  
    0,  
    [](int total, const Item& item)  
    {  
        return total + item.price;  
    });
```



● 수학 vector의 내적

```
std::vector<int> left{1, 2, 3};  
std::vector<int> right{10, 20, 30};
```

```
int result = std::inner_product(  
    left.begin(),  
    left.end(),  
    right.begin(),  
    0);
```

```
// 1 * 10 + 2 * 20 + 3 * 30
```

- 첫 범위의 길이를 기준으로 처리
- 둘째 범위에는 충분한 원소가 있어야 함

```
int matches = std::inner_product(  
    left.begin(),  
    left.end(),  
    right.begin(),  
    0,  
    std::plus<>{},  
    std::equal_to<>{});
```

- 대응 원소가 같으면 true
- true가 정수 합으로 누적됨



13.9.3 partial_sum과 adjacent_difference

```
// partial_sum  
std::vector<int> values{10, 20, 30, 40};  
std::vector<int> sums(values.size());
```

```
std::partial_sum(  
    values.begin(),  
    values.end(),  
    sums.begin());
```

```
// sums → 10 30 60 100
```

```
// adjacent_difference  
std::vector<int>  
    differences(values.size());
```

```
std::adjacent_difference(  
    values.begin(),  
    values.end(),  
    differences.begin());
```

```
// values    → 10 20 30 40  
// differences → 10 10 10 10
```

```
std::adjacent_difference(  
    sums.begin(),  
    sums.end(),  
    restored.begin());
```

- 부분 합 결과에 인접 차분 적용 시 원래 값 복원 가능





13.9.4 reduce와 transform_reduce

- `std::reduce()` → 범위 원소를 하나의 값으로 축약
- `reduce()`는 연산 순서를 바꿀 수 있음
- 재배치해도 결과가 같은 연산인지 확인 필요
- 부동소수점 합은 순서에 따라 결과가 달라질 수 있음

```
std::vector<int> values{10, 20, 30, 40};  
int total = std::reduce(values.begin(), values.end(), 0);
```

```
// accumulate  
// (((0 + 10) + 20) + 30) + 40
```

```
// reduce  
// (0 + 10) + (20 + 30) + 40 등 재배치 가능
```

```
int totalScore = std::transform_reduce(players.begin(), players.end(),  
                                       , 0, std::plus<>{}, [](const Player& player)  
                                       {  
       return player.score;  
   });
```



- 서로 다른 호출 가능한 객체를 하나의 타입으로 저장
- 저장할 호출 형태를 함수 시그니처로 지정
- 콜백을 저장하거나 실행할 함수를 나중에 결정할 때 사용

```
#include <functional>

std::function<void(int)> callback;

callback = PrintValue;
callback = Printer{};

int offset = 100;

callback = [offset](int value)
{
    std::cout << value + offset << '\n';
};

callback(10);
```

항목	내용
저장 대상	함수, 함수 객체, 람다
호출 방식	동일한 함수 호출 문법
빈 상태	가능
빈 상태 호출	std::bad_function_call 예외



구분	함수 포인터	함수자(함수 객체)	std::function
형태	함수 주소 저장	operator()를 가진 객체	호출 가능한 객체를 저장하는 래퍼
선언 예	<code>void (*callback)(int);</code>	<code>struct FucObj { void operator()(int v) }; FucObj callback;</code>	<code>std::function<void(int)> callback;</code>
저장 예	<code>callback = PrintValue;</code>	<code>Printer callback;</code>	<code>callback = PrintValue; callback = Printer{}; callback = lambda;</code>
상태저장	X	O	O
호출	<code>callback(10);</code>	<code>callback(10);</code>	<code>callback(10);</code>
주요내용	같은 시그니처 함수 저장	상태를 가진 호출 객체	함수·함수자·람다를 같은 호출 시그니처로 저장



- `std::function<void(int)>`에 저장되는 실제 타입은 서로 다름
- 공통 호출 인터페이스만 남기고 구체 타입을 숨김
- 이 방식을 타입 소거(type erasure)라고 함

```
void RunCallback(const std::function<void(int)>& callback)
{
    callback(100);
}
```

- 사용자는 저장된 대상의 타입을 몰라도 호출 가능

```
std::function<void(int)> callback = Printer{};
if (Printer* printer = callback.target<Printer>())
{
    (*printer)(10);
}
```

- 일반적인 콜백 코드는 구체 타입을 다시 확인하지 않음
- `target()`이 자주 필요하면 설계 재검토



13.10.4 콜백 인터페이스 설계

- 호출 시점
- 호출 횟수
- 예외 처리
- 스레드
- 캡처 대상의 수명
- 소유권
- 해제 방법

```
class Button
{
public:
    using ClickHandler = std::function<void()>;
    void SetOnClick(ClickHandler handler)
    {
        onClick = std::move(handler);
    }
    void Click()
    {
        if (onClick)
        {
            onClick();
        }
    }
private:
    ClickHandler onClick;
};
```

```
Button button;
int clickCount = 0;

button.SetOnClick([&clickCount]
{
    ++clickCount;
});

button.Click();
```



13.10.5 std::function의 비용과 선택 기준

- 구체 타입을 통하지 않는 간접 호출 가능
- 큰 함수 객체 저장 시 동적 메모리 할당 가능
- 복사 시 저장된 호출 대상도 복사
- 호출 경로가 성능에 민감하면 측정 필요

요구	우선 검토 방식
컴파일 시점에 호출 타입 결정	함수 템플릿, auto
일반 함수 주소만 필요	함수 포인터
다양한 복사 가능 호출 대상 저장	std::function
이동 전용 호출 대상 필요	이동 전용 콜백 래퍼
호출 비용이 매우 민감	템플릿 또는 함수 포인터 + 측정

```
std::function<void()> first = [data = std::vector<int>(1000)]  
{  
    std::cout << data.size() << '\n';  
};  
  
std::function<void()> second = first;
```



- C++20
- 반복자 쌍뿐 아니라 범위 객체를 알고리즘에 전달
- View를 조합해 처리 과정을 표현
- 기존 알고리즘을 대체하는 기능이 아니라 범위 인터페이스 확장

```
#include <algorithm>
#include <ranges>
#include <vector>
```

```
std::vector<int> values{40, 10, 30, 20};
std::ranges::sort(values);
```

```
std::sort(values.begin(), values.end());
std::ranges::sort(values);
```

```
std::list<int> values{30, 10, 20};
// std::ranges::sort(values); // 임의 접근 범위 아님
```

- `begin()`, `end()` 반복 작성 감소
- Concepts로 요구 조건 표현
- 조건이 맞지 않을 때 진단이 더 직접적일 수 있음



- View는 범위의 순회 방법이나 변환 규칙을 표현
- 원소를 소유하지 않는 가벼운 범위인 경우가 많음
- 실제 순회 시 필터와 변환 수행
- 지연 평가(lazy evaluation)

```
std::vector<int> values{1, 2, 3, 4, 5, 6};

auto evenValues = values | std::views::filter([](int value)
{
    return value % 2 == 0;
});

auto squares = values | std::views::transform([](int value)
{
    return value * value;
});
```

- 중간 결과 컨테이너를 만들지 않음
- 원본 범위의 수명과 무효화 규칙에 의존



13.11.6 View와 반환 반복자의 수명

- 많은 View는 원본 범위를 참조
- 원본이 소멸하면 View를 사용할 수 없음

```
auto MakeInvalidView()  
{  
    std::vector<int> values{1, 2, 3, 4};  
  
    return values | std::views::filter([](int value)  
    {  
        return value % 2 == 0;  
    });  
}
```

- 함수 종료 후 지역 벡터 소멸
- 반환된 View가 지역 벡터를 참조하면 유효한 원소 없음

```
using Result = decltype(std::ranges::find(std::vector<int>{1, 2, 3}, 2));  
static_assert(std::is_same_v<Result, std::ranges::dangling>);
```

- 임시 비소유 범위에서 안전하지 않은 반복자 반환을 막기 위한 **dangling**
- 이름이 있는 컨테이너는 호출 뒤에도 살아 있으므로 반복자 반환 가능



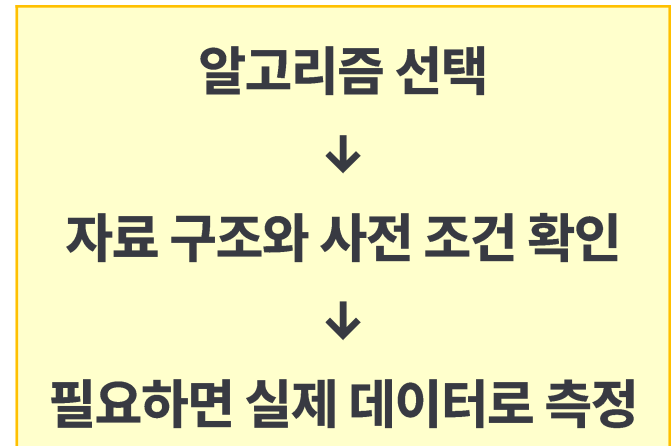
- 원하는 결과만 보지 않음
- 반복자 범주 확인
- 정렬 사전 조건 확인
- 목적지 크기 확인
- 원소 이동 비용 확인
- 메모리 사용 확인
- 반복자 무효화 확인
- 원본 범위 수명 확인
- ※ 컴파일된다는 사실만으로 논리적 사전 조건이 충족된 것은 아님

알고리즘	필요한 대표 반복자 능력
find, count, accumulate	입력 반복자
copy 목적지	출력 반복자
unique, remove	순방향 반복자 이상
reverse	양방향 반복자 이상
sort, nth_element	임의 접근 반복자





- 이론적 복잡도만으로 실제 성능을 결정할 수 없음
- 벡터의 연속 메모리와 캐시 지역성
- 리스트의 노드 할당
- 함수 객체 인라인화
- 원소 이동 비용
- 컴파일 옵션
- 데이터 크기와 연산 비율



목적	선택 후보	특징
전체 정렬	sort	범용 정렬
안정 정렬	stable_sort	동등 원소 순서 유지
상위 n개 정렬	partial_sort	앞부분만 정렬
n번째 원소와 분할	nth_element	전체 순서 불필요
정렬 범위 존재 확인	binary_search	위치 반환 없음
조건 원소 제거	erase_if	컨테이너에 맞는 제거



// 표준 알고리즘

```
bool hasNegative = std::any_of(values.begin(), values.end(),  
    [](int value)  
    {  
        return value < 0;  
    });
```

- 작업의 목적이 알고리즘 이름에 드러남

// 직접 반복문

```
int positiveCount = 0;  
int negativeCount = 0;  
long long total = 0;  
for (int value : values)  
{  
    total += value;  
    if (value >= 0)  
    {  
        ++positiveCount;  
    }  
    else  
    {  
        ++negativeCount;  
    }  
}
```

- 여러 상태를 동시에 갱신할 때
직접 반복문이 더 명확할 수 있음
- 검색, 변환, 복사, 정렬은
표준 알고리즘이 의도를 드러내는 경우가 많음



- 반복자는 컨테이너와 알고리즘 사이에서 원소 위치와 범위를 표현
- [first, last) 반 개방 구간은 빈 범위와 부분 범위를 일관되게 표현
- 반복자 범주에 따라 사용할 수 있는 알고리즘과 비용이 달라짐
- 입력과 출력 반복자는 단일 순회
- 순방향 반복자부터 다중 순회
- 임의 접근 반복자는 일정 거리 이동과 거리 계산을 상수 시간에 수행
- 연속 반복자는 원소의 연속 메모리 배치를 보장
- 반복자 무효화는 컨테이너 변경 후 기존 위치를 사용할 수 있는지 결정
- 함수 포인터, 함수 객체, 람다는 호출 가능한 객체
- 람다 표현식은 고유한 클로저 타입의 객체 생성
- 값 캡처와 참조 캡처는 수명과 소유권 의미가 다름





- 검색 알고리즘은 값을 찾거나 조건을 검사
- 정렬과 이진 탐색은 비교 규칙과 정렬 사전 조건 필요
- `remove()`, `remove_if()`, `unique()`는 컨테이너 크기를 줄이지 않음
- 실제 제거에는 `erase()` 또는 C++20 `std::erase_if()` 사용
- `accumulate()`는 왼쪽부터 순서대로 누적
- `reduce()`는 연산 순서를 바꿀 수 있음
- `std::function`은 복사 가능한 호출 대상을 하나의 시그니처로 저장하는 타입 소거 래퍼
- C++20 Ranges 알고리즘은 범위를 직접 받을 수 있음
- Projection은 원소에서 비교할 값을 분리
- View는 필터와 변환을 지연 평가
- View와 반환 반복자는 원본 범위의 수명에 의존할 수 있음
- 알고리즘은 이름보다 사전 조건, 반복자 범주, 수명과 비용 등을 고려